L21: Classes

The Droid Blueprint

Neon City Cyber Academy

Module 5: Object-Oriented Programming

🏭 Welcome to the Droid Factory

Your Mission: Design blueprints for Worker Bots

Problem: We need to make 1000 droids. Do we write 1000 different programs?

Solution: Create ONE blueprint (class), manufacture MANY droids (instances)!

🎯 What is a Class?

Class = Blueprint/Template/Recipe

Instance = Actual object created from the class

Real World:

- 🏗️ **Blueprint:** House plans
- 🏠 **Instance:** Your actual house

Code uses the same concept!

Defining Your First Class

```
class Droid:
    def __init__(self, name, battery_level):
        self.name = name
        self.battery_level = battery_level
```

Breakdown:

- `class Droid:` → Define the blueprint
- `__init__` → Constructor (runs when creating)
- `self` → Refers to the specific instance
- `self.name` → Instance variable (unique data)

🔧 The init Method

`__init__` = Initialize = Setup

Called AUTOMATICALLY when you create an instance:

```
worker = Droid('BOT-7734', 100)
```

What happens:

1. Python creates empty object
2. Calls `__init__` automatically
3. Passes `'BOT-7734'` to `name` parameter
4. Passes `100` to `battery_level` parameter

🤔 What is 'self'?

`self` = The instance being created

```
class Droid:
    def __init__(self, name, battery_level):
        self.name = name  # This droid's name
        self.battery_level = battery_level
```

When you create:

```
bot1 = Droid('ALPHA', 100)
bot2 = Droid('BETA', 80)
```

- In bot1: `self` = bot1

⚙ Adding Methods

```
class Droid:
    def __init__(self, name, battery_level):
        self.name = name
        self.battery_level = battery_level

    def greet(self):
        print(f"Greetings. I am {self.name}, Worker Unit.")
```

Methods = Functions inside a class

Always have `self` as first parameter!

🏭 Manufacturing Droids

```
# Create instances from the blueprint
worker_a = Droid('ALPHA-1', 100)
worker_b = Droid('BETA-2', 87)
worker_c = Droid('GAMMA-3', 92)

# Each has unique data
worker_a.greet()  # "I am ALPHA-1"
worker_b.greet()  # "I am BETA-2"
worker_c.greet()  # "I am GAMMA-3"
```

One class → Infinite instances!

💬 Understanding the Power

> **Without Classes:**

```
bot1_name = 'ALPHA'
bot1_battery = 100
bot2_name = 'BETA'
bot2_battery = 87
# ... Messy!
```

> **With Classes:**

```
bot1 = Droid('ALPHA', 100)
bot2 = Droid('BETA', 87)
# Clean and organized!
```

📊 Class vs Instance

Aspect	Class	Instance
What	Blueprint	Actual object
When	Defined once	Created many times
Example	`Droid`	`worker_a`
Contains	Methods & structure	Data values

🎯 Accessing Attributes

```
worker = Droid('BOT-3000', 95)

# Access attributes with dot notation
print(worker.name)           # BOT-3000
print(worker.battery_level)  # 95

# Call methods
worker.greet()
```

Dot (`.`) = "Give me this object's property/method"

More Complex Example

```
class Droid:
    def __init__(self, name, battery_level):
        self.name = name
        self.battery_level = battery_level
        self.tasks_completed = 0  # Default value

    def work(self):
        self.tasks_completed += 1
        self.battery_level -= 5
        print(f"{self.name} worked! Battery: {self.battery_level}%")

    def recharge(self):
        self.battery_level = 100
        print(f"{self.name} recharged to 100%!")
```

✅ Lesson Complete!

You Mastered:

- ✅ What classes are (blueprints)
- ✅ Creating classes with `class ClassName:`
- ✅ The `__init__` constructor
- ✅ Understanding `self`
- ✅ Adding methods to classes
- ✅ Creating multiple instances

Next: L22 - Inheritance

🏆 Challenge

Build a BattleBot class:

- Attributes: name, health, attack_power
- Methods: `attack()`, `take_damage()`, `status()`
- Create 2 bots and simulate a battle

Show the power of OOP!